Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla

Materia: Implementación de robótica inteligente

Clave: TE3002B.501

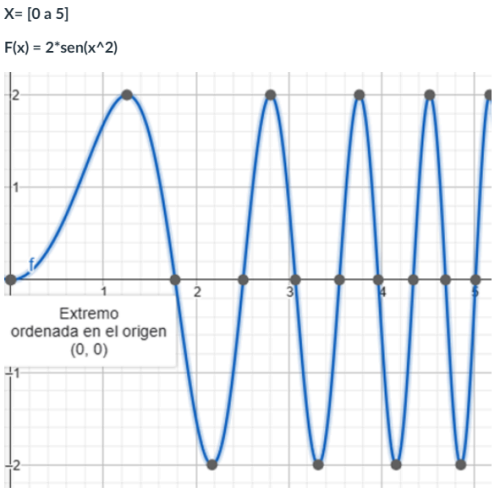
Grupo: 501

Profesores:
Rigoberto Cerino Jiménez
Juan Manuel Ahuactzin Larios
Alfredo García Suárez
Cesar Torres Huitzil

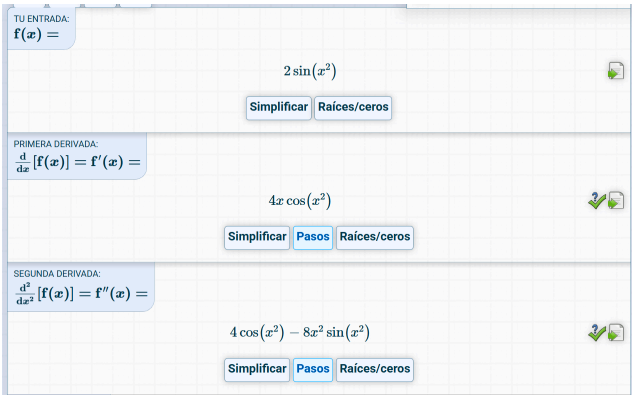
Alumnos
Omar Perez Dominguez | A01738306

21 de Abril De 2026

Trayectoria 1:



Para poder resolver la primera y segunda derivada de nuestra función lo que hicimos fue utilizar una calculadora de derivadas.



otros cambios que se realizaron que fueron importantes para que el robot logra la trayectoria fue cambiar

%%%% CÁLCULO DE VX

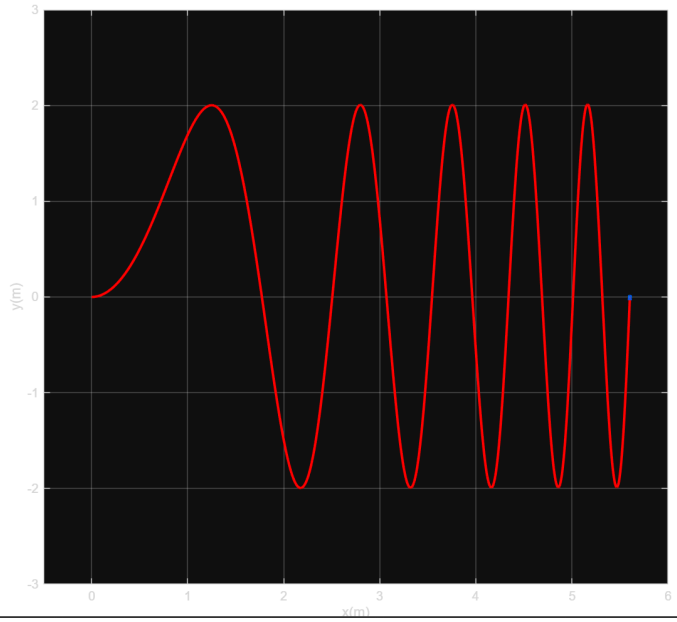
vx = sqrt(10*pi)/5;

establecemos la velocidad constante necesaria para que el Puzzlebot recorra esa distancia exacta justo cuando el cronómetro marque los 5 segundos, garantizando que la trayectoria termine completa y a tiempo.

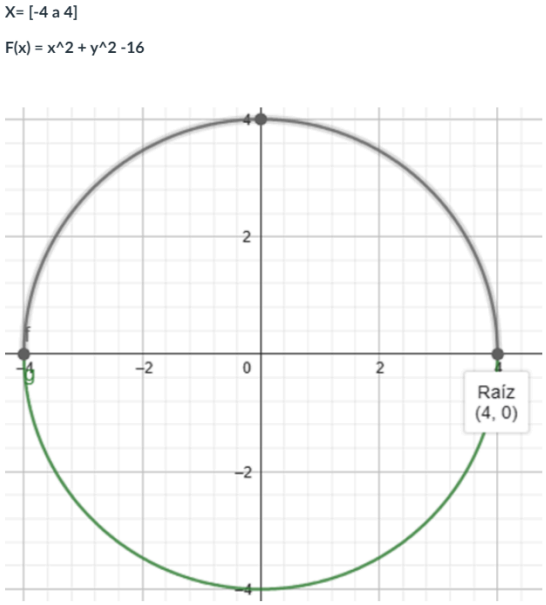
Este bloque del código realiza el mapeo necesario para que el robot siga la trayectoria deseada en tiempo real. Primero, calcula la posición actual en el eje X asumiendo un avance constante vx, como lo habíamos visto antes este era sumamente importante porque le decia al robot que tan rápido tenía que moverse. Después, obtiene la pendiente (dy_dx) y la curvatura (ddy_dxx) de la función 2 * sin(x^2) mediante sus derivadas respecto al espacio. Finalmente, aplica una transformación de coordenadas para convertir estos cambios en comandos físicos de velocidad lineal (u) y angular (w), asegurando que el robot mantenga el rumbo correcto y se oriente tangencialmente a la curva mientras avanza.

```
%%%%%%%%%%%%%%%%%%%%%%%%% VELOCIDADES DE REFERENCIA %%%%%%%%%%%%%%%%%%%%%%%%%%%%%
u = zeros(1,N);
w = zeros(1,N);

for k = 1:N
    tk = t(k);
    xk = vx * tk;

    % Función: f(x) = 2 * sin(x^2)
    % Derivadas  (respecto a x)
    dy_dx = 4 * xk * cos(xk^2);
    ddy_dxx = 4 * cos(xk^2) - 8 * (xk^2) * sin(xk^2);


    % Usamos el avance vx constante en el eje X
    u(k) = vx * sqrt(1 + dy_dx^2);
    w(k) = (ddy_dxx * vx) / (1 + dy_dx^2);
end
```

Trayectoria final



Trayectoria 2:



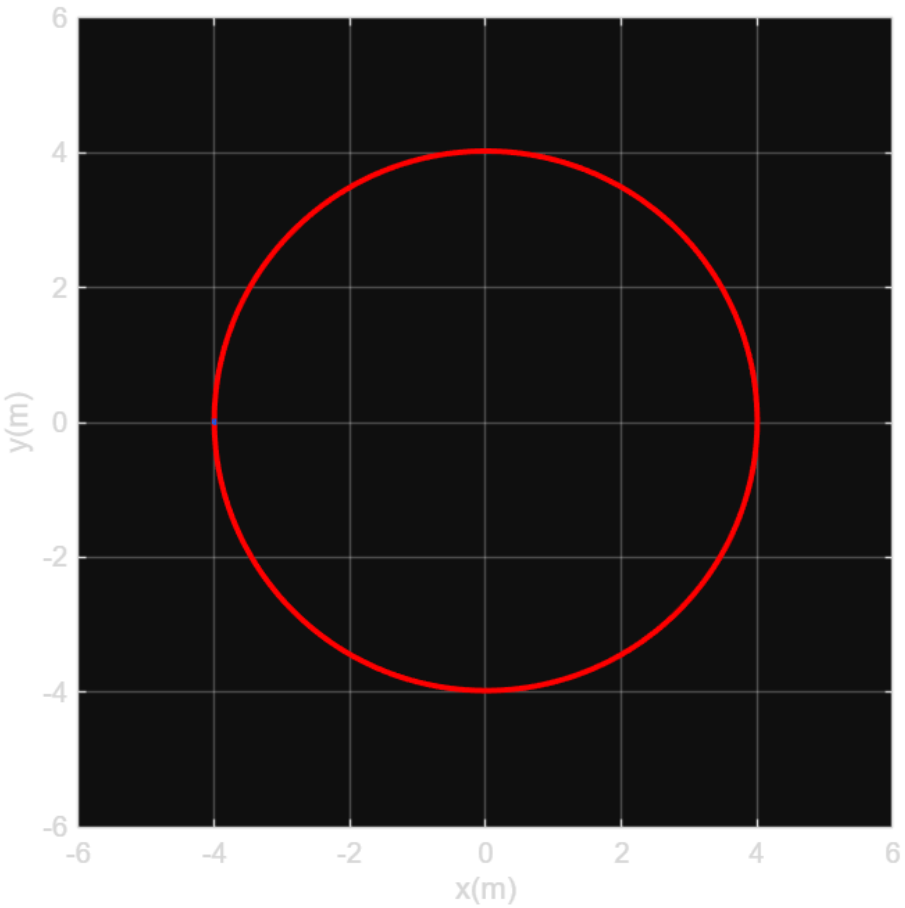
La trayectoria fue modelada utilizando un enfoque paramétrico basado en la variación temporal del ángulo θ. Esta metodología simplifica la transformación de coordenadas espaciales a comandos de velocidad lineal y angular para el chasis diferencial.

```
%%%%%%%%%%%%%%%%%%%%%%%%%% PARÁMETROS DEL CÍRCULO %%%%%%%%%%%%%%%%%%%%%%%%%%
R = 4;
omega_ref = (2 * pi) / tf;
```

R = 4: Es simplemente el tamaño. Es para que el robot sepa qué tan amplia tiene que ser la curva y no se salga del camino.

omega_ref: Para poder completar una vuelta necesitamos 2*pi radianes. Entonces, al dividir eso entre el tiempo total (tf), estamos estableciendo el tiempo que deberia tardar el robot en completar su trayectoria.

```
%%%%%%%%%%%%%%%%%%%%%%% VELOCIDADES DE REFERENCIA %%%%%%%%%%%%%%%%%%%%%%%%%%
% En un círculo perfecto, u y w son constantes
u = (omega_ref * R) * ones(1,N); % Velocidad lineal (v = omega * R)
w = omega_ref * ones(1,N);       % Velocidad angular constante
```

u: Esta es la velocidad lineal, es para saber qué tan rápido vas se va moviendo en el círculo, multiplicas la velocidad de giro por el radio (u = w * r). Con esto nos aseguramos de que el robot no se quede corto ni se pase de largo en la curva.
w: Esta es la velocidad angular (qué tan rápido gira sobre su propio eje). Como el radio del círculo no cambia, el giro es siempre el mismo, así que la dejamos constante con el valor de omega_ref que calculamos antes.

Trayectoria final